Tribhuwan University

Institute of Science and Technology

Bhairahawa Multiple Campus

A Project Report

On

Sajilo Sahayata (Disaster Reporting & Coordination System)

(In partial fulfilment of the requirement of Bachelor of Science in Computer Science & Information Technology (BSc. CSIT))

Submitted By:

Arun Neupane (30282 / 078)

Bijay Kumar Chaudhary (30287 / 078)

Sunil Paudyal (30335 / 078)

Submitted To:

Department of Computer Science and Information Technology

Bhairahawa Multiple Campus

Bhairahawa, Nepal

July, 2025

CERTIFICATE OF APPROVAL

This is to certify that the project report titled "Sajilo Sahayata" prepared by Arun Neupane (30282/078), Bijay Kumar Chaudhary (30287/078) and Sunil Paudyal (30335/078) under the supervision of Mr. Suraj Pandey in a partial fulfilment of the project of seventh semester of BSc. Computer Science and Information Technology has been well studied and approved for further development.

<table>
<tr><td colspan="2">…….
Lucky Prasad Joshi
Campus Chief
Bhairahawa Multiple Campus</td></tr>
<tr><td>……
Sunil Kumar Shah
Program Coordinator
Bhairahawa Multiple Campus</td><td>…….
Suraj Pandey
Supervisor
Bhairahawa Multiple Campus</td></tr>
<tr><td>……..
Internal Examiner</td><td>……
External Examiner
IOST: Tribhuwan University</td></tr>
</table>

SUPERVISOR RECOMMENDATION

This is to recommend that Arun Neupane (TU Exam Roll no: 30282/078), Bijay Kumar Chaudhary (TU Exam Roll no: 30286/078) and Sunil Paudyal (TU Exam Roll no: 30335/078), has carried out project work entitled "Sajilo Sahayata (Disaster Reporting & Coordination System" for the requirement in degree of B.Sc. in Computer Science and Information Technology in Project Work under my supervision.

To my knowledge, this work has not been submitted to any other degree. They have fulfilled all the requirements laid down by the Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal for the submission of the project work for the partial fulfilment of Bachelor of Science (B.Sc.) degree.

……

Suraj Pandey

Supervisor

Bhairahawa Multiple Campus

ACKNOWLEDGEMENT

We express our heartfelt gratitude to Bhairahawa Multiple Campus, Bhairahawa, for providing us with essential resources and continuous support throughout our project journey. The institution's commitment to academic excellence has deeply influenced our learning and project development.

A special thanks to our supervisor Mr. Suraj Pandey and program coordinator Mr. Sunil Kumar Shah for his outstanding guidance and prompt mentorship, which played a key role in the success of this project. We also acknowledge the valuable input and encouragement from our other respected supervisors.

We sincerely thank our classmates for their motivation and collaborative spirit, which helped us build a positive and productive environment.

Team Members:

……

Arun Neupane (30282/078)

……

Bijay Kumar Chaudhary (30287/078)

……

Sunil Paudyal (30335/078)

BSC.CSIT 7th semester

Bhairahawa Multiple Campus, Bhairahawa

ABSTRACT

Sajilo Sahayata is a real-time, multi-purpose reporting and coordination platform designed for Local Government body. It primarily aims to enhance disaster management by enabling citizens to report emergencies such as floods, landslides, and earthquakes with precise GPS coordinates and visual evidence (photos/videos). The system also facilitates broader citizen-government interaction by allowing the reporting of other community issues like accidents and garbage problems. Authorities can efficiently verify, prioritize, and respond to incidents through an integrated dashboard. The platform utilizes HTML, CSS, and JavaScript for the frontend, Node.js for the backend, and Firebase for database management. Key features include OTP authentication, anti-spam measures, real-time mapping, and a notification system. By employing agile development methodologies, the project ensures iterative progress through phases of requirement analysis, feasibility study, system design, and implementation. The system aims to reduce the gap between the government bodies and citizens. The expected outcomes are improved emergency response times, enhanced data accuracy, and strengthened citizen engagement in local governance.

Keywords: *Disaster management, Citizen reporting, E-governance, Real time alerts, Geo-spatial technology, OTP verification, User-friendly interface*

TABLE OF CONTENTS

CERTIFICATE OF APPROVAL i
SUPERVISOR RECOMMENDATION ii
ACKNOWLEDGEMENT iii
ABSTRACT iv
LIST OF ABBREVIATIONS vii
LIST OF FIGURES viii
LIST OF TABLES ix
CHPATER 1: INTRODUCTION 1
1.1. Introduction 1
1.2. Problem Statement 1
1.3. Objectives 2
1.3.1. Main Objectives 2
1.3.2. Specific Objectives 3
1.4. Scope and Limitation 3
1.4.1. Scope 3
1.4.2. Limitation 4
1.5. Development Methodology 4
1.6. Report Organization 5
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW 6
2.1. Background Study 6
2.2. Literature Review 7
CHAPTER 3: SYSTEM ANALYSIS 8
3.1. System Analysis 8
3.1.1. Requirement Analysis 8
3.1.2. Feasibility Analysis 11
3.1.3. Analysis 12
CHAPTER 4: SYSTEM DESIGN 17
4.1. Design 17
4.1.1. Database Design 17
4.1.2. Forms and Report Design 17
4.1.3. Interface and Dialogue Design 20
4.2. Algorithm Details 21
CHAPTER 5: IMPLEMENTATION AND TESTING 23
5.1. Implementation 23

5.1.1. Tools used 23
5.1.2. Implementation Details of Module 23
5.2. Testing 26
5.2.1. Unit testing 26
5.2.2. System Testing 27
5.3. Result Analysis 28
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION 29
6.1. Conclusion 29
6.2. Future Recommendation 29
REFERENCE 30
APPENDICES 31

LIST OF ABBREVIATIONS

AI – Artificial Intelligence

API – Application Programming Interface

BIPAD – Building Information Platform Against Disaster

CBDRM – Community-based Disaster Risk Management

CSRF – Cross-Site Request Forgery

CSS – Cascading Style Sheet

DDR – Disaster Risk Reduction

DFD – Data Flow Diagram

DoS – Denial of Service

ER – Entity Relationship

GPS – Global Positioning System

HTML – Hypertext Markup Language

JSON – JavaScript Object Notation

NF – Normal Form

NGO – Non-Governmental Organization

OTP – One Time Password

SQL – Structured Query Language

ST – System Testing

UI – User Interface

UNDRR – Unite Nation Office for Disaster Risk Reduction

UT – Unit Testing

XSS – Cross-Site Scripting

LIST OF FIGURES

Figure 1.5: Agile Development Method of Sajilo Sahayata 4
Figure 3.1.1: Use case Diagram of Sajilo Sahayata 10
Figure 3.1.2: Gantt Chart of Sajilo Sahayata 12
Figure 3.1.3.1: ER daigram of Sajilo Sahayata..... 13
Figure 3.1.3.2:DFD level 0 of Sajilo Sahayata 14
Figure 3.1.3.3: DFD level 1 of Sajilo Sahayata 15
Figure 3.1.3.4: DFD level 2 of Sajilo Sahayata 16
Annex 4.1.2.1: Login Page of Sajilo Sahayata 18
Annex 4.1.2.2: Signup Page of Sajilo Sahayata..... 18
Annex 4.1.2.3: Incident Reporting page of Sajilo Sahayata 19

LIST OF TABLES

Table 5.2.1.1: Test case of user login 26
Table 5.2.1.2: Test case of incident reporting (Real Image, Details required)..... 27
Table 5.2.2: System testing scenario of Sajilo Sahayata 27

CHPATER 1: INTRODUCTION

1.1. Introduction

Nepal is geographically diverse and highly susceptible to natural disasters such as earthquakes, landslides, floods, and fires. These hazards often result in substantial damage to lives, property, and infrastructure. Effective disaster response and public issue coordination require not only robust infrastructure but also efficient, real-time information exchange between citizens and government authorities. However, existing systems in Nepal are typically fragmented, slow, and reliant on traditional means such as phone calls, paperwork, and disjointed applications, which limits their effectiveness during critical emergencies.

In recent years, advancements in digital technologies have offered opportunities to transform disaster management and public service delivery. Despite several government-led initiatives and international support systems, a citizen-centric, real-time platform that supports two-way communication and geospatial coordination remains largely absent. Addressing this gap, the proposed project, Real-Time Disaster Management and Coordination System, seeks to leverage modern web technologies, cloud services, and geospatial visualization to enhance incident reporting, coordination, and response capabilities at local and national levels.

This system is designed as a real-time, web-based platform where citizens can report incidents (e.g., natural disasters, civic issues) and government agencies can manage and respond to those reports through a centralized dashboard. It includes functionalities such as real-time mapping, multimedia evidence submission, status tracking, role-based access control, and location-based alerts. The platform is scalable, accessible across devices, and inclusive for use across urban and rural communities in Nepal.

1.2. Problem Statement

Nepal, due to its complex geography, fragile infrastructure, and limited digital penetration in rural areas, faces critical challenges in disaster management and public service coordination. The current systems for disaster reporting and community issue resolution are fragmented and inefficient, relying heavily on traditional methods such as physical visits, phone calls, or separate, siloed applications. These methods often result in delays, loss of information, and uncoordinated responses during emergencies.

There is an urgent need for a centralized, real-time, and citizen-inclusive digital platform that allows timely reporting, accurate verification, and streamlined coordination among government bodies. The lack of such a system not only hampers effective emergency response but also undermines public trust and participation in local governance. Sajilo Sahayata aims to address these issues by creating an integrated digital ecosystem for disaster management and civic coordination in Nepal.

Key Problems in the Context of Nepal.

i. Fragmented and Redundant Communication Channels

- Citizens report issues via multiple disconnected means: ward offices, police stations, NGOs, and helplines.
- These parallel systems cause delays and lead to poor tracking of complaints.

ii. Absence of Real-Time Information Flow

- Many rural and urban areas lack platforms for immediate, real-time updates during floods, landslides, fires, or road blockages.
- Emergency teams often operate with outdated or second-hand information.

iii. Verification and Authenticity Issues

- In crisis scenarios, it's difficult to verify the legitimacy of user-reported incidents.
- This causes confusion, resource misallocation, and spread of misinformation (especially via social media).

iv. Low Digital Civic Participation

- A significant portion of the population remains passive due to lack of awareness or accessible tools for engagement.
- Existing apps/platforms rarely allow public tracking of issue status or offer feedback mechanisms.

v. Weak Inter-Agency Coordination

- Agencies duplicate efforts or overlook key areas due to poor synchronization.
- Lack of interoperability between municipal systems, disaster response units, and local law enforcement.

1.3. Objectives

1.3.1. Main Objectives

- To design and develop an integrated, real-time disaster management and civic coordination system tailored for Nepal’s socio-geographic context.

- To bridge the gap between citizens and authorities by enabling efficient, transparent, and timely communication during emergencies and community-level issues.
- To comprise web, incorporating modern geospatial and communication technologies to ensure reliability, scalability, and inclusiveness.

1.3.2. Specific Objectives

i. Develop a user-centric reporting interface:

- Allow real-time submission of incident reports with GPS coordinates
- Support multimedia attachments (images, videos) for context-rich reporting.
- Enable easy categorization of incidents by type (natural disasters, infrastructure issues, etc.).

ii. Create a dynamic incident management dashboard for government agencies:

- Visualize reports on a real-time interactive map using geospatial technologies.
- Prioritize and assign tasks to appropriate response teams with tracking capabilities.

iii. Enhance citizen engagement and trust:

- Provide transparent issue tracking with status updates for reported incidents.
- Support multiple languages (including Nepali) for inclusiveness across regions.

1.4. Scope and Limitation

The scope and limitation of this project is given below:

1.4.1. Scope

i. Incident Reporting (Web & Mobile)

- Real-time submission with location and media.

ii. Auto Location Detection:

- Identifies user's local government body automatically.

iii. Report Tracking:

- Filter and track reports by status (*Pending*, *In Progress*, *Completed*).

iv. Interactive Map:

- Leaflet-based maps with clustering and shortest path routing.

v. Admin Dashboard:

- Tools for report verification, status updates, and analytics.

vi. Multilingual Support:

- Language switcher between English and Nepali.

vii. Scalability:

- Designed for nationwide expansion beyond initial deployment in Pokhara.

1.4.2. Limitation

i. Media Restriction:

- Images must be captured within 5 seconds to reduce fake reports.

ii. Limited Language Options:

- Only English and Nepali are currently supported.

iii. No Offline Mode:

- Requires internet access for reporting and data access.

1.5. Development Methodology

The software development model chosen for the Sajilo Sahayata is the Agile Development Model. Agile is selected due to its flexibility in accommodating changing requirements, which is crucial in a disaster management system aimed at local government. The iterative and incremental nature of Agile allows for continuous updates and testing, ensuring timely delivery of essential features while maintaining system reliability. Additionally, Agile promotes regular communication with stakeholders, enabling effective alignment with local government protocols and integrating real-time feedback effectively.

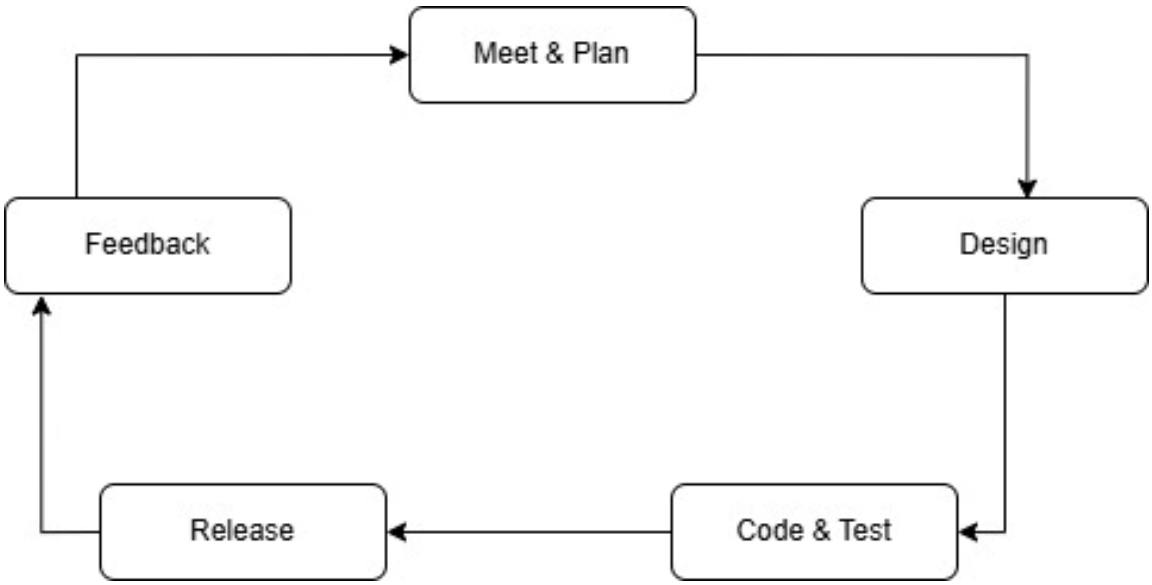


Figure 1.5: Agile Development Method of Sajilo Sahayata

1.6. Report Organization

This report is structured into six comprehensive chapters that systematically document the entire development process of the project "Sajilo Sahayata: Disaster Reporting and Coordination System." Each chapter provides insight into specific phases of the system development life cycle:

- **Chapter 1: Introduction**
  Presents an overview of the project, including the background, problem statement, objectives, scope and limitations, the development methodology used, and the organization of this report.
- **Chapter 2: Background Study and Literature Review**
  Discusses foundational theories, concepts related to disaster management systems, and reviews existing literature and similar platforms relevant to the project.
- **Chapter 3: System Analysis**
  Covers requirement analysis (both functional and non-functional), feasibility study (technical, operational, economic, and schedule), and system modelling using appropriate structured or object-oriented techniques.
- **Chapter 4: System Design**
  Describes the system architecture, design diagrams (component, activity, sequence), and wireframes. It also includes the design of algorithms and logic flows implemented in the system.
- **Chapter 5: Implementation and Testing**
  Provides detailed information about the technologies and tools used, the implementation of core modules, unit and system testing procedures, test cases, and result analysis.
- **Chapter 6: Conclusion and Future Recommendation**
  Summarizes the achievements of the project, reflects on limitations, and suggests future improvements and scalability options.

Additional sections such as References, Appendices, and Screenshots are included at the end to provide supporting documentation, source code snippets, and logs of interactions with supervisors.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1. Background Study

Disaster management requires coordinated efforts to prepare for, respond to, and recover from natural or man-made disasters. Nepal, being highly prone to earthquakes, floods, and landslides, faces major challenges due to delayed responses and fragmented communication channels in traditional disaster response systems.

Digital technology offers a powerful solution to improve emergency response. As highlighted by Unite Nation Office for Disaster Risk Reduction (UNDRR) [1] and the World Bank, centralized digital platforms with real-time alerts, geospatial mapping, and citizen input can significantly enhance disaster preparedness and reduce loss of life and property. Features like location tracking, live updates, and centralized dashboards help authorities make faster, more informed decisions.

Community-based disaster risk management (CBDRM) has also gained attention for promoting citizen participation in local preparedness and reporting. This approach emphasizes the need for inclusive tools such as multilingual mobile apps that engage users across Nepal's diverse regions, ensuring more reliable and resilient disaster response systems. [2]

Some Terminologies Related to Sajilo Sahayata

- **Real-Time Reporting:** Enables users to immediately submit disaster incidents or community issues with accurate time and location stamps.
- **Geospatial Mapping:** Utilizes maps to visualize incident locations, helping responders understand the spatial distribution and severity of reported events. [3]
- **Incident Verification Workflow:** A process where submitted reports go through a screening or approval phase to confirm authenticity before action is taken.
- **Status Tracking System:** Allows users and authorities to monitor the progress of reported issues—from pending to completed.
- **Multilingual Support:** The app supports multiple languages (currently English and Nepali) to ensure accessibility and usability for a broader population.

- **Local Body Routing:** Based on GPS location, each user's report is directed to the respective local authority, enhancing response time and accountability.

This background provides the foundation for the Sajilo Sahayata system, a modern platform designed to improve how Nepal prepares for and responds to disasters and community challenges.

2.2. Literature Review

Nepal has taken several steps toward improving disaster risk reduction and digital governance. However, a fully integrated, real-time, citizen-centric disaster coordination system is still lacking. Below are the key systems.

- **BIPAD Portal (Building Information Platform Against Disaster)**
  Led by the Government of Nepal under the Ministry of Home Affairs, BIPAD is a national platform aimed at disaster risk reduction, data integration, and visualization. While it supports data-driven decision-making, it primarily focuses on institutional-level data and lacks robust citizen reporting capabilities. [1]
- **Nepal Disaster Risk Reduction (DRR) Portal**
  This portal consolidates national disaster statistics and risk mapping but does not support interactive user engagement or real-time reporting from the public. [2]
- **Ushahidi Platform**
  A global open-source crowdsourcing tool used for crisis mapping and data visualization. Although not Nepal-specific, it has been adapted during crises in various countries. Ushahidi shows the value of public-sourced data but lacks direct integration with government response frameworks in Nepal. [3]

CHAPTER 3: SYSTEM ANALYSIS

3.1. System Analysis

System analysis is essential for defining the requirements and understanding how Sajilo Sahayata will function. It focuses on both functional requirements such as incident reporting, map visualization, and role-based access and non-functional requirements like performance, security, usability, and scalability to ensure reliability during emergencies.

A feasibility study confirms the technical suitability of using React.js, Node.js, Firebase, and Leaflet, as well as the economic and operational viability of the system. The project timeline is detailed in a Gantt chart covering all development phases.

Visual aids like flowcharts, use case diagrams, and activity diagrams are provided to illustrate system processes and user interactions, supporting better design and implementation planning.

3.1.1. Requirement Analysis

The requirements analysis is broken down into two sections namely function requirements and non-functional requirements and they are discussed below.

i. Functional Requirement

For Citizen

- User Registration and Authentication:

a. Register using a mobile number or email address.

b. Secure OTP-based login for identity verification.

c. Edit and manage user profile (name, contact info, location).

- Incident Reporting:

a. Submit incidents with:

i. Text descriptions.

ii. GPS-based location tagging.

iii. Photo and video uploads.

iv. Categorization (e.g., earthquake, flood, fire, road blockage, health emergency).

b. View history and status of submitted reports.

c. Receive real-time notifications, alerts, and updates on reported issues.

- Map Interaction:

a. View live incident map with markers and status indicators.

b. Filter incidents based on category, severity, or location.

c. Zoom in/out for locality-specific or nationwide visualization.

For Government Officials

- Authentication and Access:

a. Manage permissions and roles for different government users.

- Incident Management:

a. Access real-time dashboard of reported incidents.

b. Verify incoming reports using supporting evidence and GPS data.

c. Track and update progress of each incident (pending, under review, resolved).

- Communication:

a. Send broadcast alerts or incident-specific messages to affected citizens.

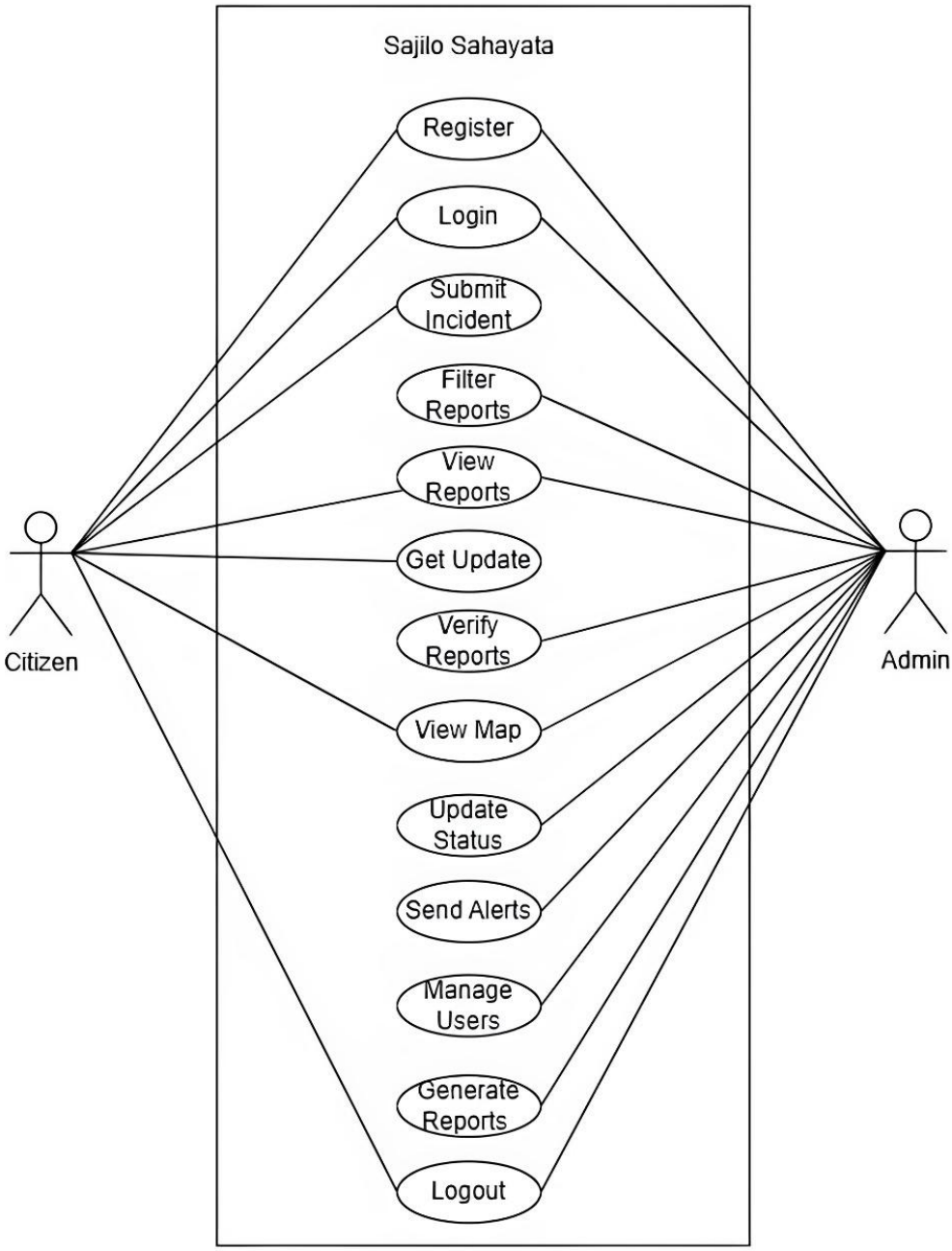


Figure 3.1.1: Use case Diagram of Sajilo Sahayata

ii. Non-functional Requirements

a. Performance:

- All system interactions (page loads, form submissions, map renders) should respond fast under normal load.
- System should support thousands of concurrent users, especially during disaster peaks.

b. Security:

- End-to-end encryption of sensitive data (e.g., user data, location, media).

- Protection against web threats: SQL injection, XSS, CSRF, DoS attacks.
- Periodic security audits and role-based access controls.

c. Usability:

- Clean, intuitive interface with simplified navigation.
- Full responsiveness across devices (desktop, tablet, mobile).
- Multi-language support (initially Nepali and English) for wider accessibility.

d. Reliability:

- 99.9% uptime with redundant servers and fallback systems.
- Real-time data syncing and conflict resolution mechanisms.
- Regular system health checks and failure recovery procedures.

e. Scalability:

- Modular architecture to add new features or integrate with external systems.
- Cloud-native deployment for auto-scaling based on demand.
- Designed to support a nationwide rollout across municipalities and provinces.

3.1.2. Feasibility Analysis

i. Technical Feasibility Analysis

- The technologies chosen ReactJs for the frontend, Node.js for the backend, for real-time data handling, and open street maps for geospatial visualization—are mature, widely supported, and proven effective for real-time applications.
- Open-source libraries and frameworks reduce licensing and long-term maintenance costs.
- The development team is already proficient in the selected technologies, reducing the learning curve and accelerating implementation.
- The platform is deployable on common cloud environments like Firebase Hosting, Vercel, Render ensuring high availability with minimal infrastructure overhead.

ii. Economic Feasibility Analysis

- Estimated operational cost: NPR 1,500–3,000/month (covers hosting, database usage under moderate scale, and domain/email services) and almost no or minimal expense for production as we have been using open source and free libraries.
- The system will reduce the cost of delayed disaster response, improve coordination, and minimize losses, making it economically viable.

- Long-term potential for government adoption or external funding through NGOs, disaster response donors, or innovation grants.

iii. Operational Feasibility Analysis

- The system is accessible through modern web browsers and can be extended into a mobile PWA or native app if needed.
- Designed for intuitive use by both citizens (reporters) and officials (responders).
- Short training workshops or user manuals can be provided for local government bodies and volunteers.
- Real-time updates, notification systems, and automated maps improve operational responsiveness during emergencies.

iv. Schedule Feasibility Analysis

Sajilo Sahayata						
Weeks / Work	1-2	3-4	5-6	7-8	9-10	11-12
Planning						
Design						
Coding						
Test & Debug						
Implementation						
Documentation						

Figure 3.1.2: Gantt Chart of Sajilo Sahayata

3.1.3. Analysis

System analysis is a crucial phase in software development that involves understanding and modelling how the system should behave. For Sajilo Sahayata, the analysis follows a structured approach using both Entity-Relationship (ER) Diagram and Data Flow Diagram (DFD) to represent how data flows within the system and how different entities interact with one another.

- **Data Modelling using Entity Relationship (ER) Diagram**

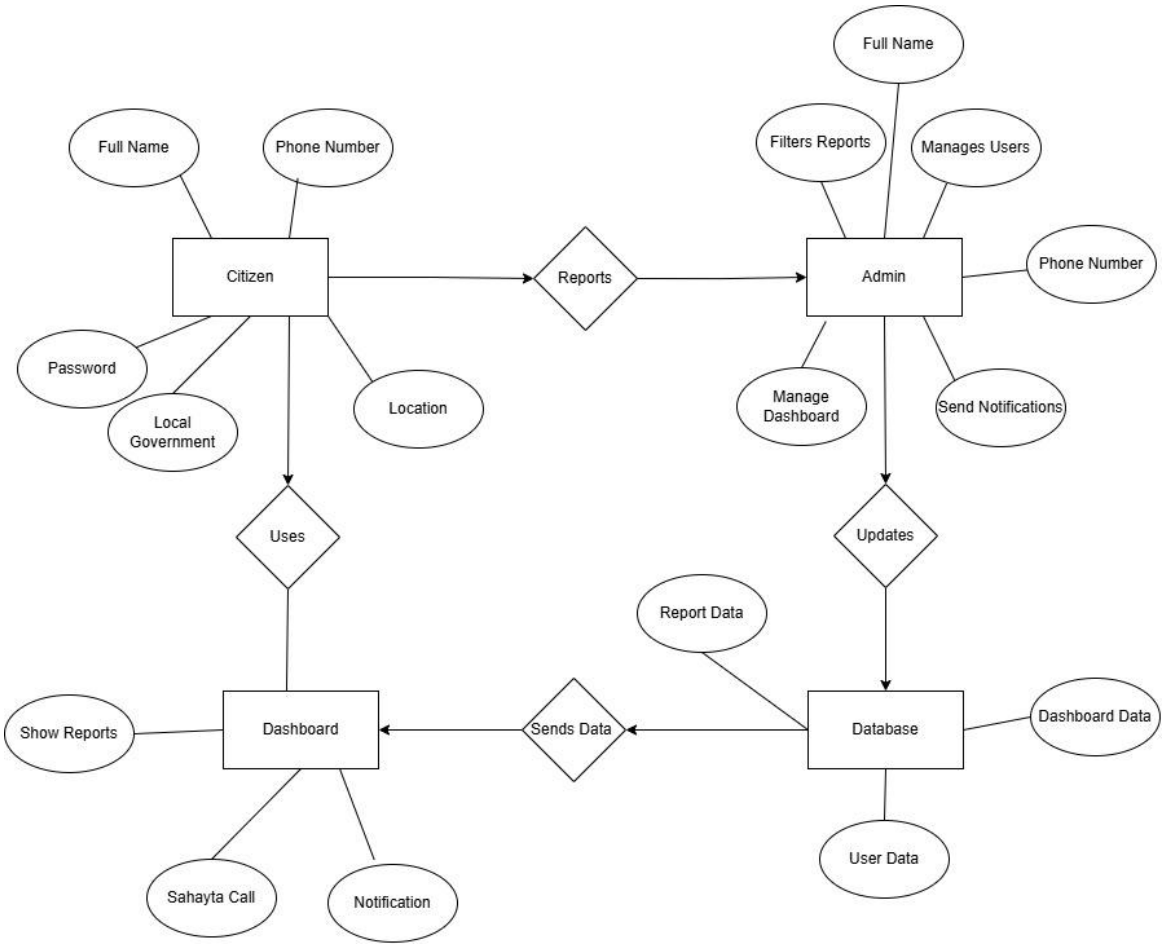


Figure 3.1.3.1: ER diagram of Sajilo Sahayata

- **Process Modelling using DFD**

i. DFD Level – 0

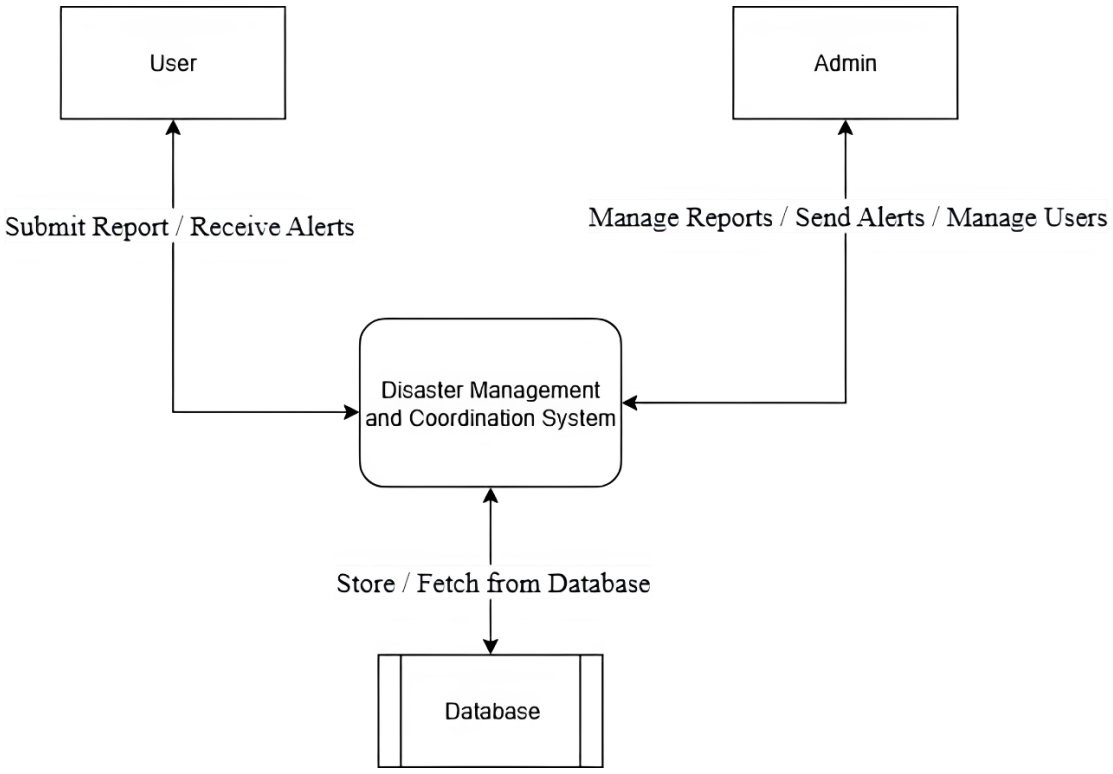


Figure 3.1.3.2: DFD level 0 of Sajilo Sahayata

ii. DFD Level – 1

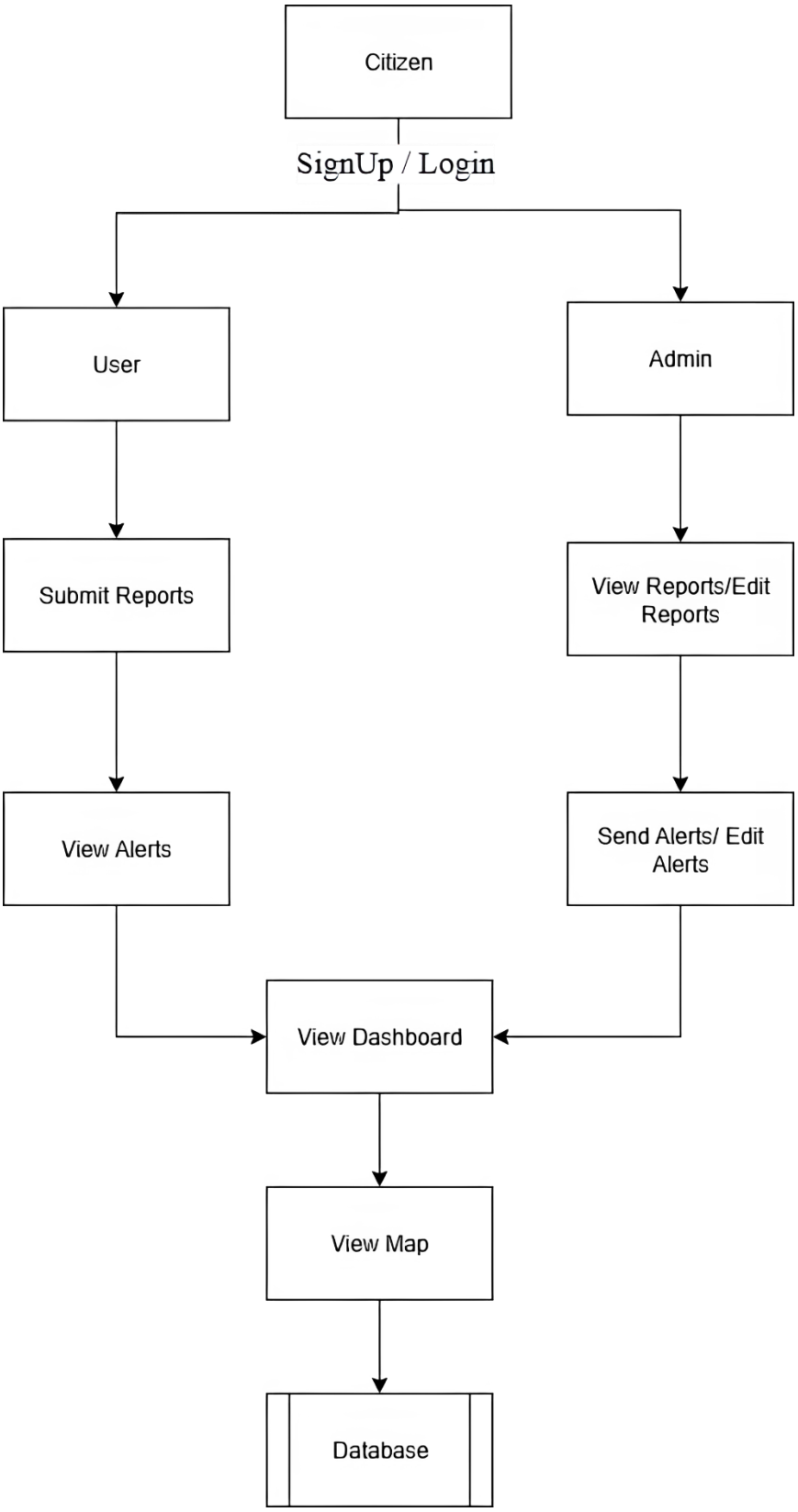


Figure 3.1.3.3: DFD level 1 of Sajilo Sahayata

iii. DFD Level – 2

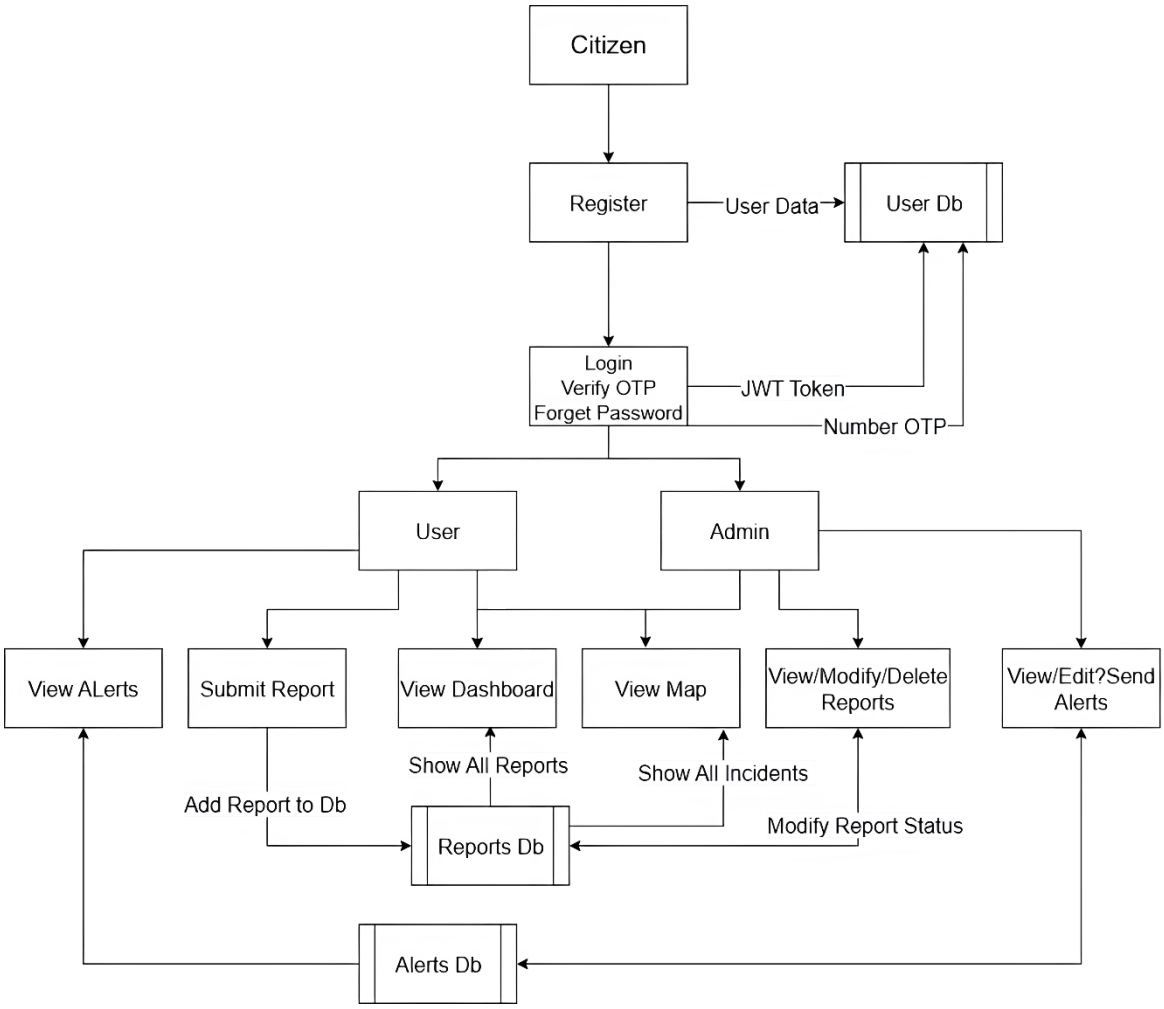


Figure 3.1.3.4: DFD level 2 of Sajilo Sahayata

CHAPTER 4: SYSTEM DESIGN

4.1. Design

In the structured design approach, we focus on defining the system's architecture by transforming the data analysis (especially from ER diagrams and DFDs) into a concrete implementation plan. This chapter includes database design, forms and reports design, as well as interface and dialogue design.

4.1.1. Database Design

The database design was derived from the ER diagram and normalized to ensure data integrity and minimize redundancy. The design adheres to the principles of the **Third Normal Form (3NF)** to avoid anomalies and support efficient data storage and retrieval.

- The Users table stores citizen information such as full name, phone number, location, and authentication credentials. Each user is uniquely identified by a primary key.
- The Incidents table records all reported events with fields like incident type, description, GPS-based location, media attachments (images/videos), timestamps, and status (e.g., reported, under review, resolved).
- The Officials table manages government personnel who interact with the system. It includes fields like full name, contact number, role and assigned permissions.
- The Dashboard stores real-time status updates and summary data for quick access by users and officials. It interacts with both incident and user data.
- The Notifications and Sahayata Call tables support alert broadcasting and emergency communication features, helping users stay informed during disasters.

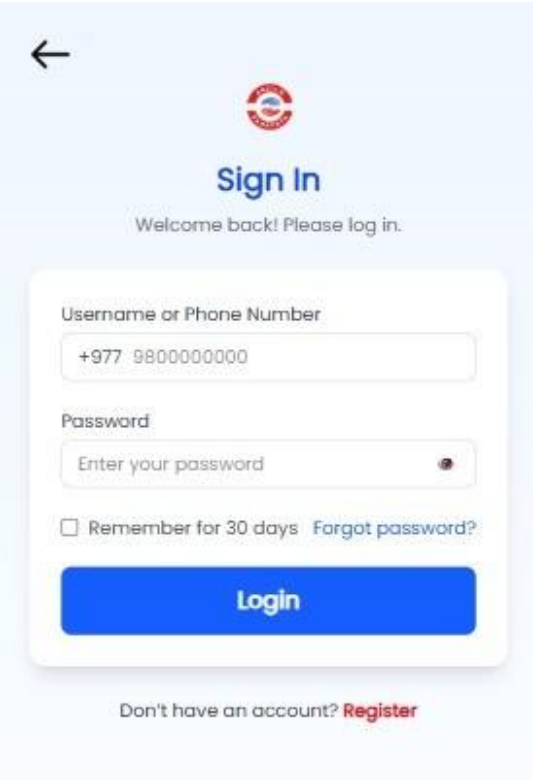
All relationships between entities are implemented using foreign key constraints to maintain referential integrity. This structured approach ensures a scalable, maintainable, and secure database foundation for the system.

4.1.2. Forms and Report Design

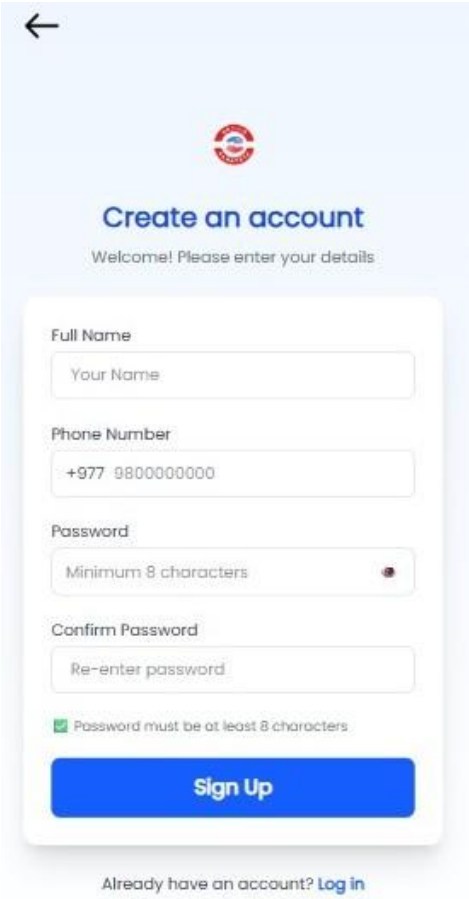
Forms and reports are designed to facilitate smooth data entry and meaningful output.

i. **Forms**:

- User Registration/Login Form: Allows users to register and securely log in.

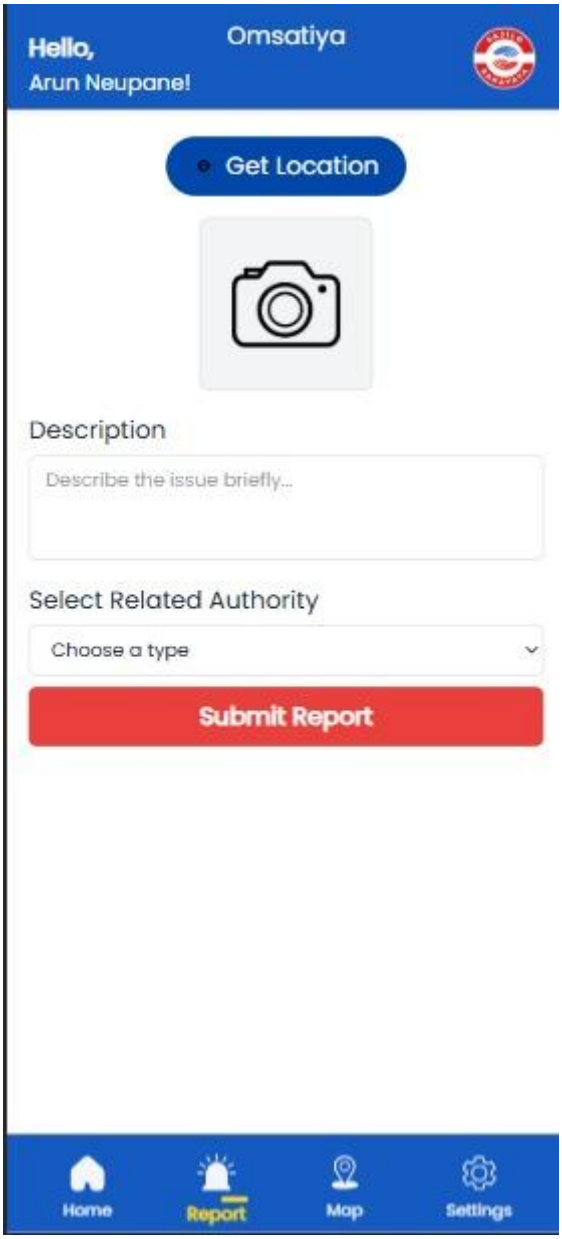


Annex 4.1.2.1: Login Page of Sajilo Sahayata



Annex 4.1.2.2: Signup Page of Sajilo Sahayata

- Incident Report Form: Enables users to describe incidents with photo, location, and category.



Annex 4.1.2.3: Incident Reporting page of Sajilo Sahayata

ii. **Reports**:

- **Incident Report Summary**: Displays verified and pending incidents.
- **User Activity Logs**: Track user interactions and submission history.
- **System Analytics**: Graphical representation of most common disasters by region or category.

4.1.3. Interface and Dialogue Design

The Interface and Dialogue Design focuses on creating a smooth and intuitive interaction between the user and the system. The system is built using ReactJs, TailwindCSS, MongoDB, and NodeJs with clear navigation, structured layouts, and role-based dashboards for Farmers, Customers, and Admins. Each user type interacts with the system through personalized dashboards and dedicated forms.

i. **Interface Design**

The system provides a user friendly and consistent user interface with the following features.

- **Role-Based Dashboard**

  The system consists role-based dashboard. Based on the role i.e. User and Admin they are directed to their respective dashboard.

  User Dashboard: The user can view the reports, filter reports, add reports with proper detail of the incident. They can also view their profile and edit their profile; they can change the language like English to Nepali and Nepali to English. Also, the user can make an emergency call to the respective department of incident handler.

- **Navigation Menus**

  It consists of navigation menu on each dashboard that helps user to navigate from one page to another page.

- **Form**

  It also consists of form like report incident, login, signup, etc. where user can fill-up the form for respective purpose.

- **Visual Elements**

  There is various icon used in this system where each icon has their own meaning and made for their own purpose. There is icon used for resembling each type of incident.

ii. **Dialogue Design**

The dialogue design ensures the user receive important notification and message either from system or from admin.

- **Notification**

  The user can receive alert notification for the admin that helps user to be pre-prepared for the upcoming natural disaster.

4.2. Algorithm Details

In the Real-Time Disaster Management and Coordination System, incident reports submitted by citizens are dynamically categorized and sorted into different statuses such as Pending, Working, Solved, and Reported. To optimize performance when rendering large volumes of such records, especially on the dashboard or filtered map view, the system uses the Timsort algorithm a hybrid sorting algorithm derived from merge sort and insertion sort.

Purpose:

Timsort is well-suited for this system due to its:

- High efficiency on partially sorted data: common in incident queues
- Stable sort behaviour: preserves original report order when keys match
- Proven performance in real-world applications: used in Python and JavaScript engines

Description:

Timsort works by:

- Dividing the array of incidents into smaller chunks called runs.
- Sorting each run using insertion sort.
- Merging the sorted runs using merge sort.
- This hybrid approach benefits the application where incidents are frequently updated and often partially sorted by creation time or status.

Application in the system:

In the system's incident dashboard:

- Each report object contains fields like status, timestamp, location, and type.
- When users filter incidents by status (e.g., "Pending"), the frontend:

a. Extracts the relevant subset.
b. Applies Timsort to order them by timestamp (most recent first).
c. Ensures fast rendering and response even with hundreds of entries.

Pseudo Code

```
function sortIncidentsByStatus(incidents):
runs = splitIntoRuns(incidents)
for each run in runs:
insertionSort(run)
while multiple runs remain:
```

mergeRuns(runs)

return mergedSortedIncidents

Time and space complexity

Operation	Time complexity	Space Complexity
Best Case	O (n)	O (n)
Average case	O (n logn)	O (n)
Worst Case	O (n logn)	O (n)

This makes Timsort an ideal choice for sorting the live and historical incident data used by the system.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1. Implementation

5.1.1. Tools used

5.1.1.1. Programming Language

Frontend

- **ReactJS**: A popular JavaScript library used to build dynamic and responsive user interfaces. It allows the creation of reusable UI components, improving development efficiency and user experience.
- **Tailwind CSS**: A utility-first CSS framework that enables rapid UI design directly in markup, ensuring consistent styling with minimal custom CSS.

Backend

- **Node.js**: A JavaScript runtime environment that allows running server-side code. It is efficient and suitable for building scalable, real-time applications.
- **Express.js**: A minimal and flexible Node.js web application framework that provides robust features for building APIs and web services.

Database

- **MongoDB**: A NoSQL database that stores data in flexible, JSON-like documents. Ideal for handling unstructured data such as incident reports with media and location tags.

5.1.2. Implementation Details of Module

i. User Login Interface

This module provides the user login functionality. It includes a mobile-number-based login form with OTP-style formatting and password input. Upon successful validation, the system simulates a login using predefined credentials and redirects the user to the dashboard. It also includes real-time error feedback, password visibility toggling, “Remember Me” option, and language localization support via react-i18next

Key Features:

- Phone number and password validation
- Error handling and multilingual support
- UI feedback for success/failure
- Navigation to password recovery and signup modules

ii. User Registration Interface

This module allows new users to create an account. It includes fields for full name, phone number, password, and confirmation. Validation rules ensure correct formatting, password strength, and consistency. Upon validation, the system simulates a login and redirects to OTP verification (for future integration). It also utilizes Zustand for temporarily storing registration steps and react-i18next for localization.

Key Features:

- Form validation with user feedback
- Conditional navigation to OTP screen
- Password visibility toggle
- Responsive and accessible UI

iii. Citizen Dashboard (Incident Overview)

This module is the main dashboard page for citizens. It fetches and displays recent incident reports along with government-issued alerts. Incidents are sorted and filtered using status types (e.g., working, pending, solved). The dashboard includes geolocation-based labeling using reverse geocoding via OpenStreetMap APIs. Users can also navigate to a map view of incidents or initiate emergency calls via a prominent SOS button.

Key Features:

- Real-time display of incidents and alerts
- Reverse geocoding with caching
- Filtering and sorting using Timsort logic
- Interactive and animated UI with framer-motion

iv. Incident Reporting Form

This module allows citizens to report emergencies and civic issues. It captures the user's current geolocation, media (image or video), incident description, and type/category. The form validates the inputs and submits the data to the backend using multipart/form-data. On successful submission, the module triggers confetti animations and UI feedback to confirm submission.

Key Features:

- Real-time GPS location capture
- Multimedia upload with camera/gallery detection
- Category tagging and validation
- Animated feedback on submission

- Integration with backend API for data submission

v. **Interactive Disaster Map**

This module displays all reported incidents on a map using Leaflet and marker clustering. It supports: Real-time incident rendering, Filtering by category (fire, flood, etc.), Focusing on specific incidents or user location, Integration with external directions (Google Maps).

Key Features:

- Map clustering for high-density data visualization
- Dynamic filtering by incident types
- Custom animated markers with live geolocation
- Focused fly-to animations and routing support

vi. **Unauthorized Access Handler**

This is a simple fallback page shown to users who try to access restricted areas without proper authentication. It redirects them to the dashboard homepage after a 5-second delay.

Key Features:

- Access denial messaging
- Auto-redirect with countdown
- Prevents unauthorized entry to sensitive components

vii. **Admin Dashboard & Analytics**

This is the main admin analytics panel. It visualizes reports, user stats, and alerts using charts (Pie, Bar, Line) via Chart.js. Admins can monitor the overall system health at a glance, including: Report status breakdown User role distribution Alerts over time

Key Features:

- Real-time analytics
- Multiple chart visualizations
- Data fetched from APIs
- Insight into user and incident patterns

viii. **User Management Panel**

This module gives admins control over user accounts. Admins can: View all registered users, Promote/demote roles (admin/user), Search and filter by name/email/phone, Delete user accounts.

Key Features:

- Role-based access control
- Live search filter

- User promotion/demotion actions
- Confirmation modal for sensitive actions

ix. Alert Broadcast Manager

This module enables admins to create, edit, and delete real-time alerts that are broadcasted to all users. These alerts are displayed on user dashboards and the map.

Key Features:

- Alert creation and editing via modal
- Time-stamped and location-tagged alerts
- Immediate frontend visibility post-submission
- Deletion and confirmation handling

5.2. Testing

Testing is a crucial phase in the development lifecycle of *Sajilo Sahayata*, ensuring that the system performs as expected under various conditions. Both unit and system testing were carried out to validate the functionality, usability, and performance of the platform.

5.2.1. Unit testing

Unit testing involves testing individual components or modules of the application to ensure that each part works correctly in isolation. The main focus was on:

- **Incident Reporting Module**: Validated inputs like location, description, and media uploads.
- **Language Switcher**: Verified whether switching languages updates the interface dynamically.
- **Status Filter**: Checked filtering functionality in the homepage (Pending, Working, Completed).

Table 5.2.1.1: Test case of user login

Test Case ID	Description	Input	Expected Output	Actual Result
UT-LOGIN-01	Login with valid credentials	Phone: 9811420975, Password: 88888888	User is logged in	Login success
UT-LOGIN-02	Invalid phone number	Phone: 98000, Password: Test@123	Error: Invalid phone number	Error shown

UT-LOGIN-03	Invalid password	Phone: 9800000000, Password: wrongpwd	Error: Incorrect password	Error shown
UT-LOGIN-04	Missing credentials	Phone: , Password:	Error: Fields required	Validation triggered

Table 5.2.1.2: Test case of incident reporting (Real Image, Details required)

Test Case ID	Description	Input	Expected Output	Actual Result
UT-REPORT-01	Submit valid report	Image taken from camera, valid incident details	Report submitted successfully	Report saved
UT-REPORT-02	Image captured more than 5 seconds ago	Image field empty, valid details	Error: Please enter live photo	Error shown
UT-REPORT-03	Missing incident details	Image valid, detail field empty	Error: Details required	Validation error
UT-REPORT-04	Missing image	Details provided, no image	Error: Image required	Validation error

5.2.2. System Testing

System testing is performed after unit and integration testing to ensure that the entire system works as expected in a real-world environment. It validates the overall behaviour of the application and ensures all modules and components function together properly.

In the Sajilo Sahayata system, system testing was conducted to evaluate the major functionalities such as user registration, login, incident reporting, real-time map display, and admin dashboard features. The goal was to ensure that the system met the specified requirements and performed reliably across different scenarios.

The following table highlights some of the key system-level test cases:

Table 5.2.2: System testing scenario of Sajilo Sahayata

Test Case ID	Test Scenario	Input	Expected Output	Actual Output
ST001	User Login	Valid phone number and password	User logged in successfully	As expected

ST002	Invalid Login	Wrong password	Error message shown: "Invalid password"	As expected
ST003	Report Incident	Text, category, location, and image	Incident reported successfully and shown on map	As expected
ST004	Image upload time check	Image upload from gallery	Error message: "Please upload recent image"	As expected
ST005	View all reports (map view)	User opens map	All reported incidents displayed with correct icons	As expected
ST007	Multi-language switch	User selects Nepali	App displays content in Nepali	As expected

5.3. Result Analysis

The testing phase of Sajilo Sahayata demonstrated that the core functionalities perform reliably and meet the project requirements. Unit testing confirmed that individual modules such as user login, incident reporting, and map integration operate correctly and handle errors gracefully. System testing verified that the integrated platform allows smooth interaction between citizens and government officials, with real-time data updates and notifications working as intended.

The image upload restriction (capturing images using camera) successfully prevents fraudulent reporting, improving overall data quality. The map's clustering and shortest path features enhance usability and provide clear visualization for decision-making.

Overall, the system is responsive, user-friendly, and scalable for future expansion, showing strong potential to support effective disaster management and community coordination in Nepal.

CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATION

6.1. Conclusion

Sajilo Sahayata addresses critical gaps in disaster management and civic issue reporting in Nepal by providing a real-time, integrated platform for citizens and local authorities. The system improves communication, speeds up response times, and enhances transparency through features like location tracking, incident categorization, and status updates. Testing confirms that the application is stable and functional, offering a practical solution for local governments.

The current implementation lays a strong foundation for a reliable disaster management tool tailored to Nepal's unique challenges.

6.2. Future Recommendation

- Develop a dedicated mobile app with offline capabilities and push notifications for better accessibility.
- Implement AI-powered verification to reduce false or duplicate reports.
- Expand language support to include more local languages and dialects.
- Integrate directly with emergency services, healthcare providers, and NGOs for coordinated response.
- Conduct community training and awareness programs to increase participation and trust.
- Explore opportunities for collaboration with government and international disaster relief organizations.

These improvements will strengthen the system's effectiveness and help build a more resilient, prepared, and responsive community.

REFERENCE

[1] "Unite Nation Office for Disaster Risk Reduction (UNDRR)," [Online]. Available: https://www.undrr.org. [Accessed 13 May 2025].

[2] "cbdrmplatform.org," Community-based disaster risk management , [Online]. Available: https://www.cbdrmplatform.org.

[3] "Spyrosoft," [Online]. Available: htttps://spyro-soft.com. [Accessed 10 Jun 2025].

[4] "BIPAD," [Online]. Available: https://bipadportal.gov.np. [Accessed 13 May 2025].

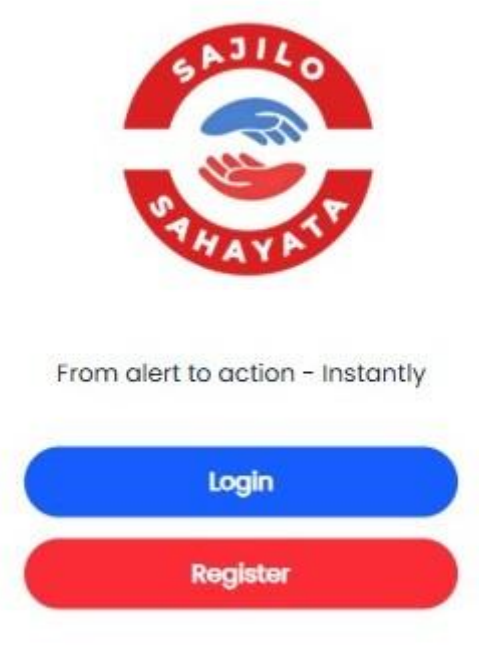
[5] "Nepal Disaster Risk Reduction Portal," [Online]. Available: https://ddrportal.gov.np. [Accessed 13 May 2025].

[6] "Ushahidi," [Online]. Available: https://www.ushahidi.com. [Accessed 13 May 2025].

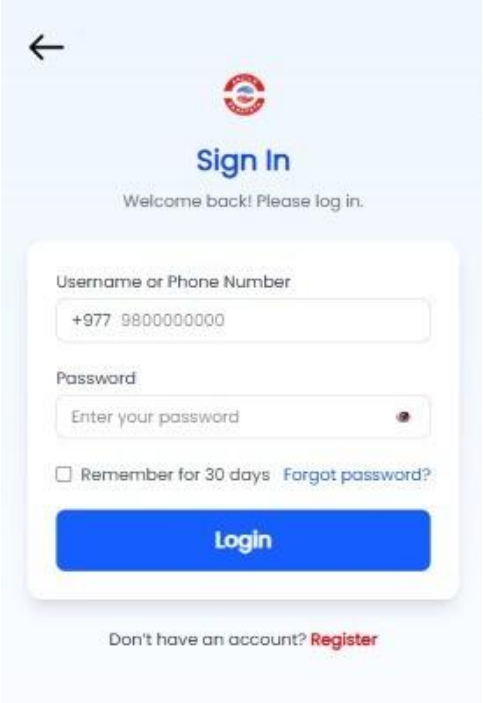
APPENDICES

Screenshots

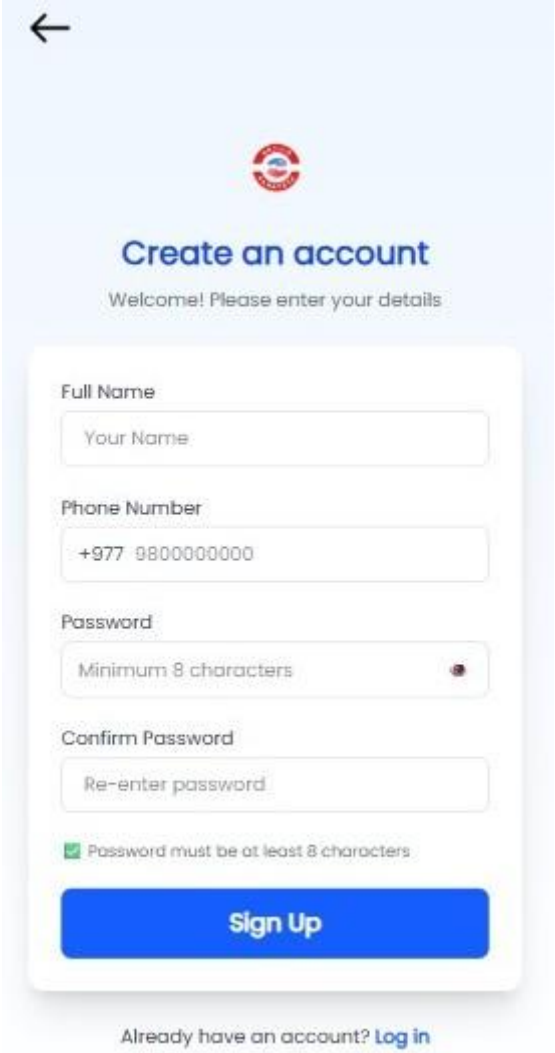
i. Welcome page



ii. Login page



iii. Register Page

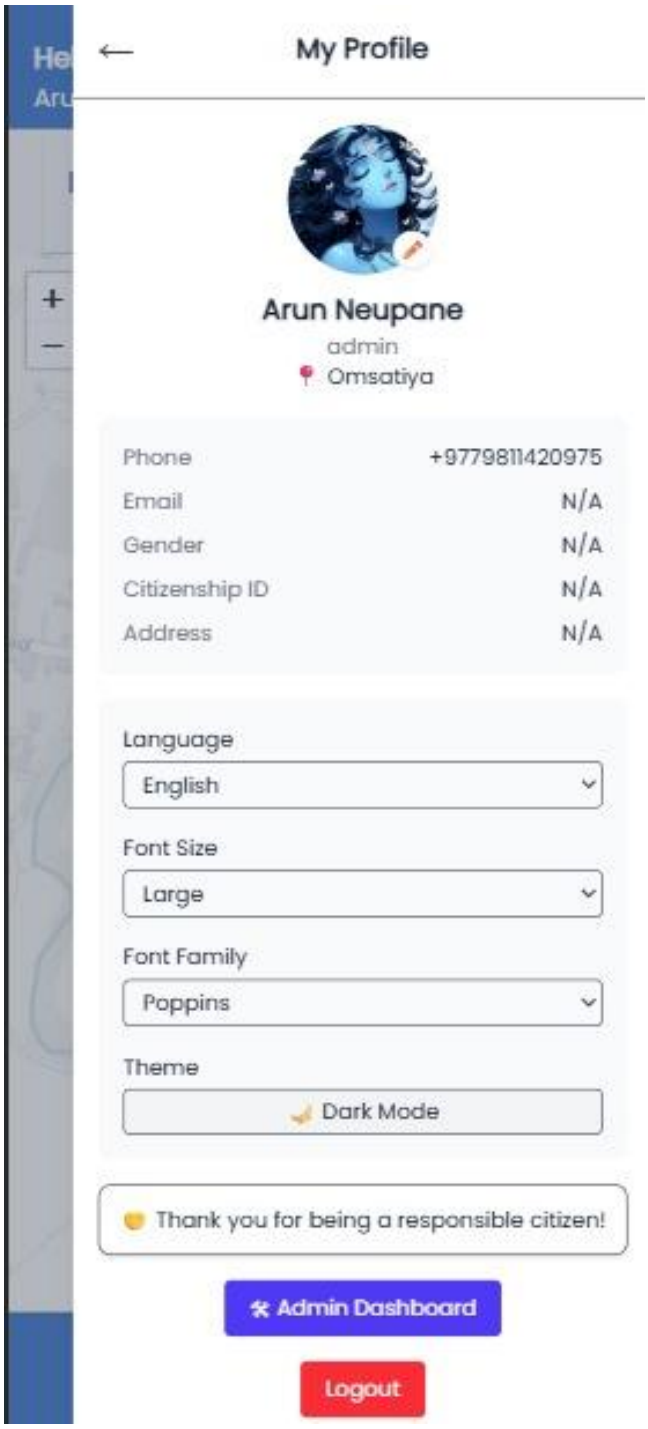


iv. Phone number verification page

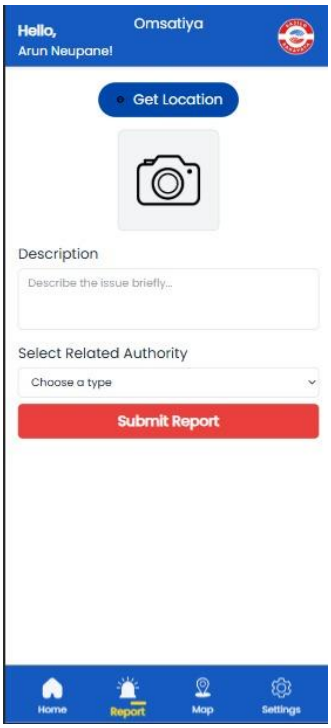
v. Forgot password page

vi. Home Page

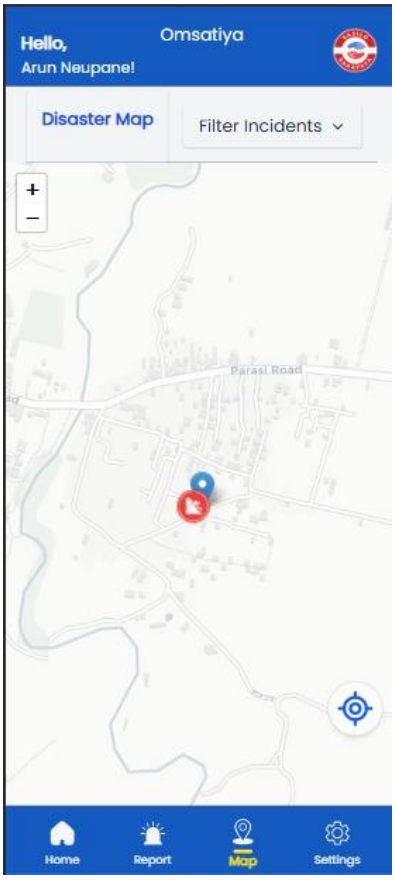
vii. My Profile page



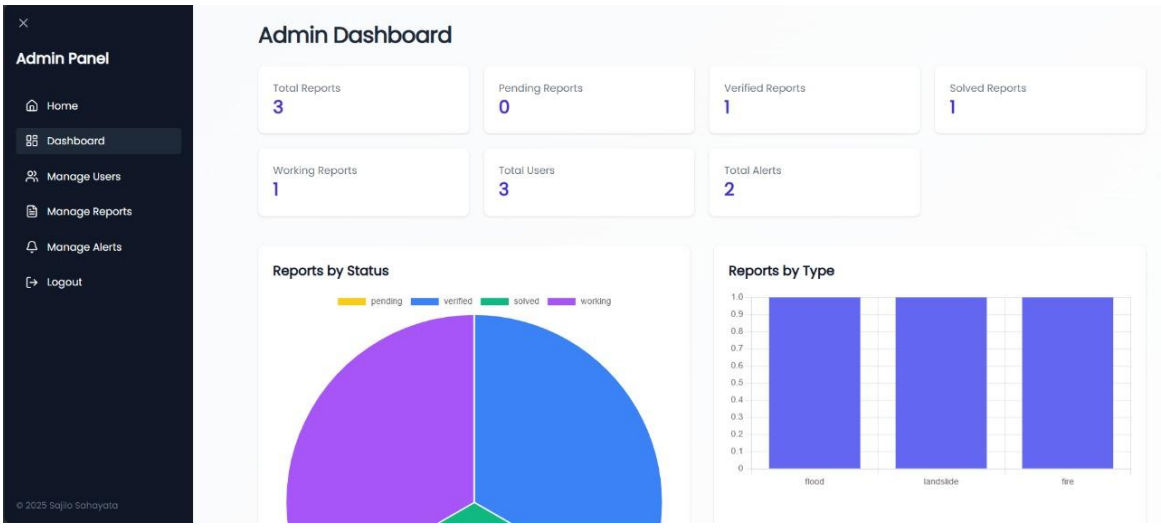
viii. Incident Reporting page



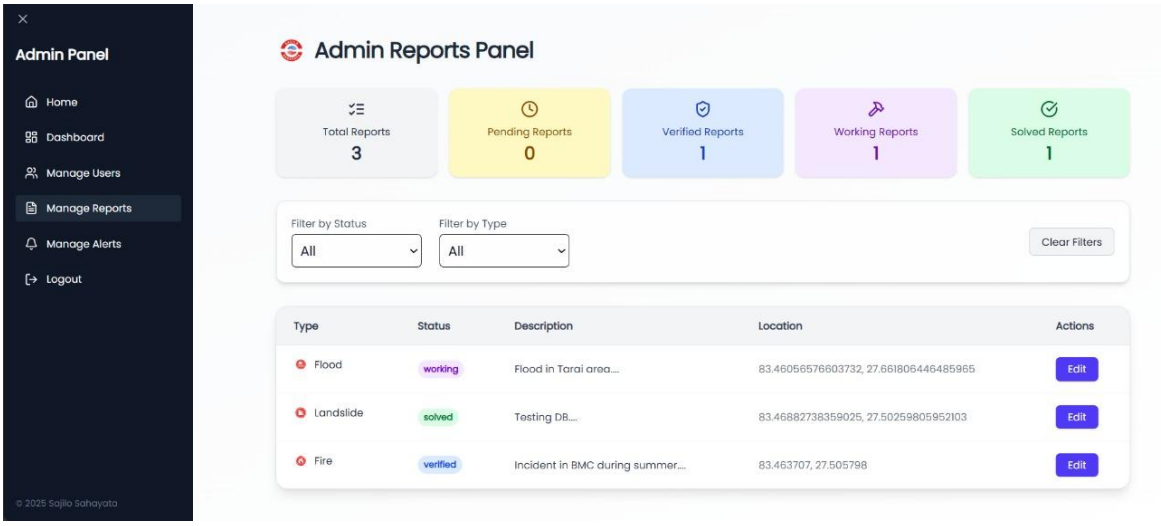
ix. Map



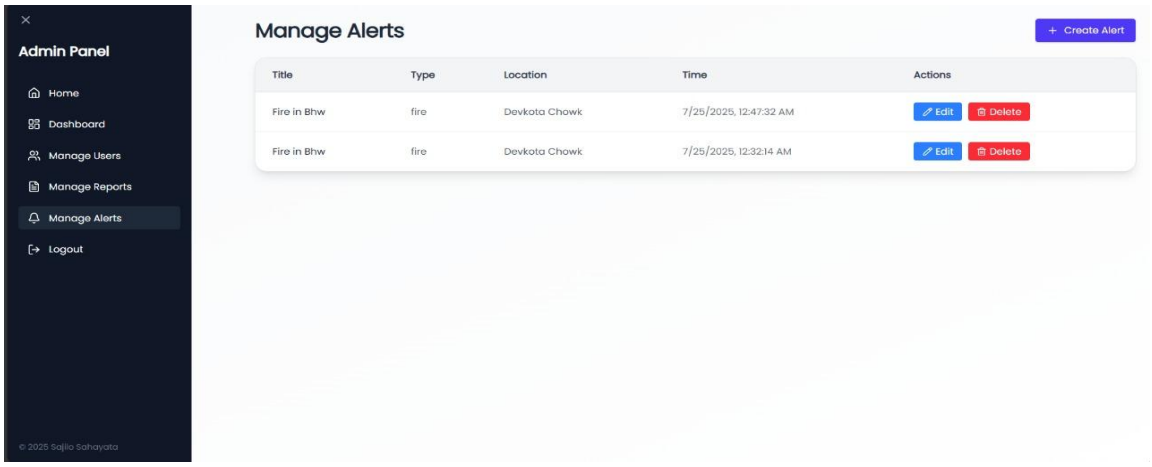
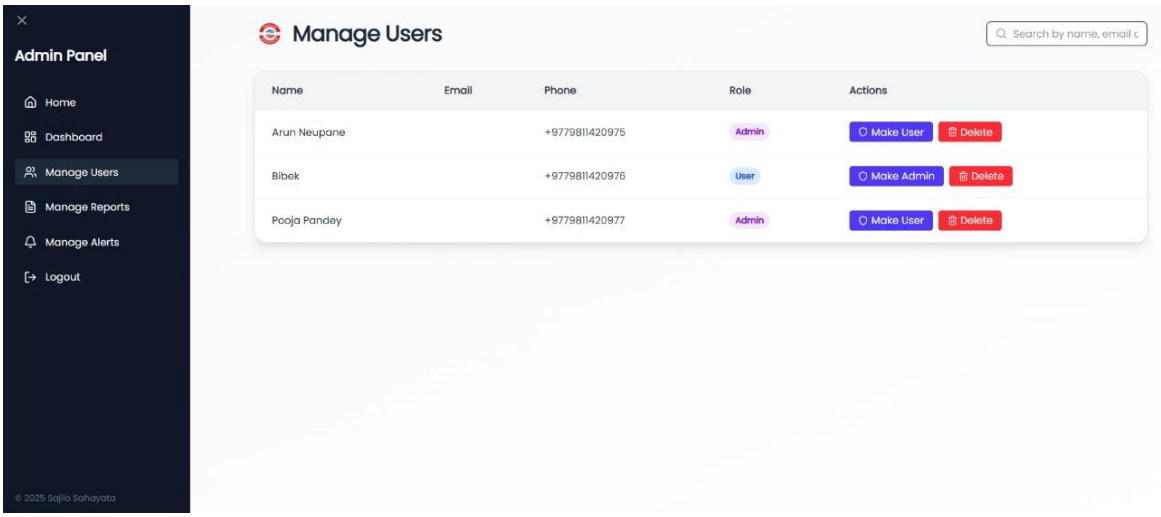
x. Admin Dashboard



xi. Admin Reports Panel



xii. Manage Alerts



xiii. Manage users